

1 RideFlow Operations and Handoff Guide

Version: Full-stack private milestone

Initial administrator: njengastephen112@gmail.com

1.1 Purpose

This guide explains how to operate RideFlow after the project is moved from the Manus workspace to a GitHub repository and a personal computer. It covers the project structure, local development, authentication, database and file storage, administrator controls, fare configuration, signup notifications, payments, backups, and the steps required before accepting real rides.

RideFlow is currently a private full-stack application. Its authentication, database, storage, fare, and notification foundations are implemented, but real payment collection, driver payouts, maps, live location, legal review, and production operations still require completion before launch.

1.2 What was implemented

Capability	Current state
React frontend and Express/tRPC backend	Implemented
Manus OAuth authentication	Connected through the full-stack template
User profiles and role-aware onboarding	Implemented
Private S3-compatible file storage	Implemented for profile photos and driver documents
Driver license, insurance, and vehicle-document uploads	Implemented with authentication gating and file validation
Nairobi server-side fare calculation	Implemented with database-backed rules
Visible 5% RideFlow commission	Implemented in quote and driver breakdowns
Admin fare settings and ownership transfer	Implemented for verified admin users
New-signup owner notification	Implemented through the owner notification channel
Card payments, M-Pesa Daraja, driver payouts, maps, live tracking	Not yet production-integrated

1.3 Moving the source to GitHub

From the RideFlow Management UI, open **Settings** → **GitHub** → **Export code to a new repository**. Use a repository name such as `rideflow`, choose the correct GitHub owner, and select private visibility while the product is still being developed. Do not commit `.env` files, API keys, OAuth secrets, database credentials, payment keys, or storage credentials.

If you prefer a ZIP backup, use **Code** → **Download** in the Management UI and upload the archive to Google Drive. GitHub is the better primary source-control location because it records history and makes it easier to work from a PC.

1.4 Setting up on a PC

Install Node.js 22 or a current supported LTS version, Git, and pnpm. Clone the repository, open a terminal in the project directory, install dependencies, and start the development server.

```
git clone https://github.com/YOUR_GITHUB_ACCOUNT/rideflow.git
cd rideflow
pnpm install
pnpm dev
```

Open the local URL printed by the development server. Do not hardcode a port in application code. The project uses React, Vite, Express, tRPC, Drizzle, MySQL/TiDB, and Vitest.

Before making a change, create a branch. After changing code, run the checks below.

```
pnpm check
pnpm test
pnpm build
```

Commit in small, readable changes and push the branch to GitHub. Use pull requests for changes that affect payments, authentication, fare calculations, admin privileges, or database schemas.

1.5 Environment configuration

The deployed environment supplies the built-in Manus variables. A local PC environment must receive equivalent values through a secure `.env` file that is never committed. The most important values include `DATABASE_URL`, `JWT_SECRET`, `VITE_APP_ID`, `OAUTH_SERVER_URL`, `VITE_OAUTH_PORTAL_URL`, the built-in Forge API URL and key, and the project storage configuration.

RideFlow also uses the following application setting:

Variable	Purpose
<code>RIDEFLOW_ADMIN_EMAIL</code>	Initial admin email used during authentication bootstrap

The current value is `njengastephen112@gmail.com`. If you change it, update the deployment secret/configuration and verify that the new account signs in before removing the old admin. Never put passwords or provider secrets in TypeScript, JSX, a PDF, GitHub, or screenshots.

1.6 Signing in as the initial admin

Sign in through Manus OAuth with the account associated with `njengastephen112@gmail.com`. The first matching user is promoted to the admin role by the server-side user upsert logic. The browser must complete the OAuth flow; the frontend must not contain a password-based admin shortcut.

If the admin panel does not appear, confirm that the account email exactly matches the configured value, confirm that the account has signed in at least once, and inspect the server logs for authentication errors.

1.7 Admin control room

Open **Profile** while signed in as an admin. The **Admin Control Room** allows you to change Nairobi fare rules without editing source code.

Setting	Meaning
Base fare	The starting KSh amount for a quote
Per km	Distance rate multiplied by the route distance
Per minute	Time rate multiplied by estimated ride duration
Safety fee	Transparent safety/support amount shown to the rider
Commission	Platform percentage, stored as basis points and shown in the fare breakdown

The new values apply to new server-side fare quotes. Existing quotes should remain traceable to the values used when they were created. Keep a minimum fare and validate values before saving. Do not use the admin panel to create hidden surge pricing; any future dynamic pricing should be disclosed to riders before booking.

1.8 Changing the administrator later

The new administrator must first sign in to RideFlow so a user record exists. In the current Admin Control Room, enter the verified email under **Transfer admin ownership** and confirm the transfer. The server demotes the previous admin, promotes the target user, and updates the owner and signup-notification email in the admin settings record.

Treat admin transfer as a high-risk operation. Confirm the target email carefully, transfer only to a trusted account, enable two-factor authentication at the identity-provider level, and test the new admin account before closing the old session.

1.9 Signup notifications

When the server creates a user record for the first time, it calls the built-in owner notification helper once. Later OAuth logins do not send duplicate signup alerts. The notification includes the new user's role, name when available, email when available, and the configured notification recipient.

The built-in owner notification channel is not the same as a customer-facing email system. Before launch, add a transactional email provider if you need messages delivered directly to an arbitrary mailbox, and add retry, delivery status, and bounce handling.

1.10 File storage

Profile photos and driver verification documents are stored in private object storage. The database stores metadata such as purpose, owner, storage key, MIME type, size, and review status. The application validates allowed MIME types and upload sizes before writing metadata.

Keep license and insurance documents private. Only the relevant account and authorized operations staff should receive signed access URLs. Add malware scanning, document review, retention rules, deletion workflows, and audit logging before driver onboarding becomes production-grade.

1.11 Fare and commission model

The current Nairobi demo model uses database-backed rules with a base fare, distance rate, time rate, minimum fare, safety fee, and a 5% platform commission. The server calculates the quote; browser JavaScript is not trusted for financial values.

For each quote, RideFlow separates the rider total, driver earnings, and platform commission. The ledger uses append-only application helpers, and the codebase exposes no update or delete procedure for ledger entries. The current TiDB database engine does not support the attempted trigger-based immutability mechanism, so production hardening should add a provider-supported append-only or audit strategy before real settlement.

The current competitor research is directional rather than a universal market truth. Public Kenya fare reports vary by city, time, vehicle type, fuel prices, route, and promotion. Do not advertise a blanket KSh 50 reduction or copy another company's fare without validating unit economics, driver earnings, taxes, payment fees, and regulatory obligations.

1.12 Payments still required

Before real rides, integrate one payment provider end to end. For cards, evaluate Stripe Connect and its marketplace onboarding, payment, refund, dispute, and payout model. For Kenya, evaluate M-Pesa Daraja with approved business credentials, STK Push, callback validation, reversals, reconciliation, and any supported business payout flow.

Keep provider keys server-side. Use idempotency keys, signed webhook verification, payment-state transitions, refund handling, payout reconciliation, and a financial operations report. Never store raw card numbers in RideFlow.

1.13 Backups and recovery

Keep three copies of the source: the private GitHub repository, a downloaded archive in Google Drive, and a local PC checkout. Back up database schema and migration files separately from database data. Confirm that object-storage files and their database metadata can be restored together.

Test restoration before launch. A backup that has never been restored is not a verified backup. Record who can restore data, how credentials are rotated, and how deleted or legally requested data is handled.

1.14 Security checklist

Use unique identity-provider credentials, two-factor authentication, protected environment variables, least-privilege admin access, HTTPS, server-side authorization, rate limits, webhook signature validation, file-type and file-size validation, audit logs, error monitoring, and database backups.

Do not commit `.env`, passwords, OAuth tokens, payment secrets, private documents, or customer exports. Rotate any credential that has been pasted into chat, screenshots, GitHub issues, or public logs.

1.15 Recommended launch sequence

1. Confirm legal entity, operating country, currency, driver classification, terms, privacy policy, and insurance requirements.
2. Complete authenticated driver onboarding and manual document review.
3. Integrate maps, route estimates, live location, realtime chat, notifications, and safety escalation.
4. Integrate one payment provider and reconcile payments, refunds, tips, and payouts.

5. Add admin audit logs, fraud controls, support tools, incident handling, and monitoring.
6. Run a closed pilot with test payments and verified drivers.
7. Publish only after the owner has reviewed the production configuration and explicitly confirmed visibility.

1.16 Troubleshooting

If the app does not start, run `pnpm install`, confirm the Node and pnpm versions, and inspect the server logs. If authentication loops, verify the OAuth portal URL, callback URL, session cookie settings, and the current browser login state. If fare changes do not appear, confirm that the admin update succeeded and generate a new quote; existing quotes should not be silently rewritten. If uploads fail, confirm authentication, allowed MIME type, size limit, storage configuration, and metadata write access.

1.17 Ownership and support model

After the GitHub handoff, the project owner is responsible for identity-provider access, GitHub access, database access, storage access, payment accounts, legal compliance, backups, and production incident response. Keep the PDF guide with the private repository and Google Drive backup. Update this guide whenever the schema, provider, admin process, or deployment process changes.